

GIT-APE - TRACK 2 - DEPLOY LIKE A PRO

Deploy Like a Pro

Multi-resource Azure with security, cost, and drift built in

60 minutes hands-on for engineering teams

Security gate

Cost transparency

CAF naming

Drift detection



AI-powered cloud deployment

9 slides

60 minutes

Engineers

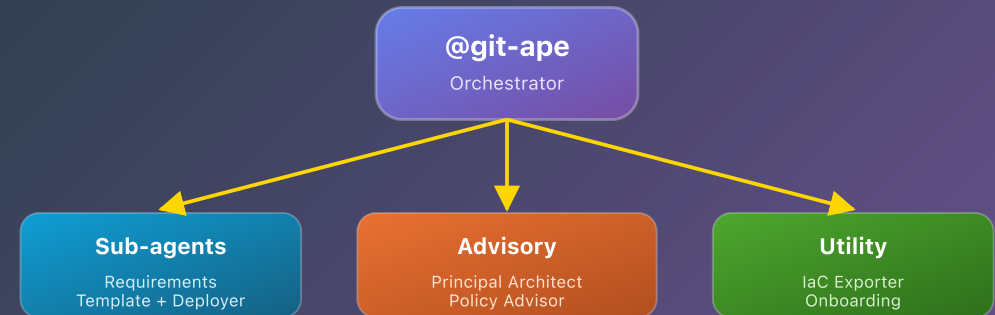
Git-Ape Architecture

8 agents, 33 skills, 4 workflows

The orchestration model

- **Orchestrator** (@git-ape) coordinates the full pipeline
- **Sub-agents** do the work: Requirements, Template, Deployer
- **Advisory** review templates: Principal Architect (WAF), Policy Advisor (CIS/NIST)
- **Utility** handle peripheral tasks: IaC Export, Onboarding
- **Skills** are invoked at every stage - naming, security, cost, testing

Agent topology



33 Skills · 4 Workflows

naming · security · cost · integration testing · drift

8 agents working together on every deployment

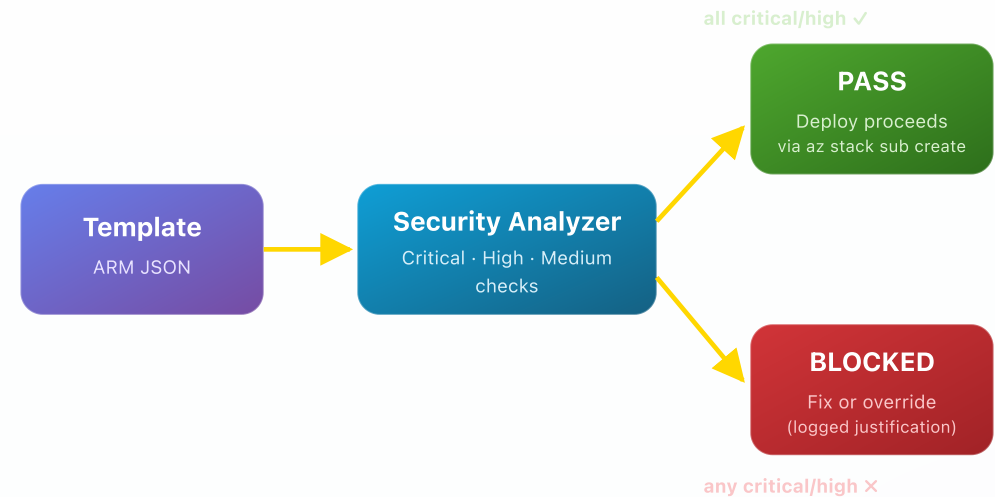
Security-First Approach

Deployment stops if security fails

What is enforced on every deployment

- **Blocking security gate** - Critical and High findings prevent deployment
- **Managed identities** everywhere - no connection strings, no shared keys
- **HTTPS-only**, TLS 1.2 plus, FTP disabled, AAD-only SQL auth
- **Key Vault references** for all secrets in app settings
- **OIDC federated identity** in CI/CD - zero stored secrets

Gate behaviour



CAF Naming Conventions

Every resource named correctly, automatically

The rules

- **Cloud Adoption Framework** naming enforced
- Pattern: `{type}-{project}-{env}-{region}`
- Validated for **length, characters, uniqueness**
- Naming conflicts caught **before deployment**

No deploy-time surprises

Consistent across teams

Live examples

Function App

`func-orderapi-dev-eastus`

Key Vault

`kv-orderapi-prod-eus` (shortened region -
24-char limit)

SQL Server

`sql-orderapi-prod-eastus`

Cost Transparency

See what you pay before you deploy

How the estimate works

- **Azure Retail Prices API** - real pricing, not averaged estimates
- **Per-resource monthly** cost breakdown
- Compare **dev vs prod SKUs** side by side
- Surfaced **before** the deploy commits

No billing surprises

Real numbers

Sample estimate (Web App + SQL + KV)

\$12.40 /mo

Dev SKUs: App Service B1, SQL Basic, KV Standard

\$184.20 /mo

Prod SKUs: P1v3, S2, KV Premium, App Insights

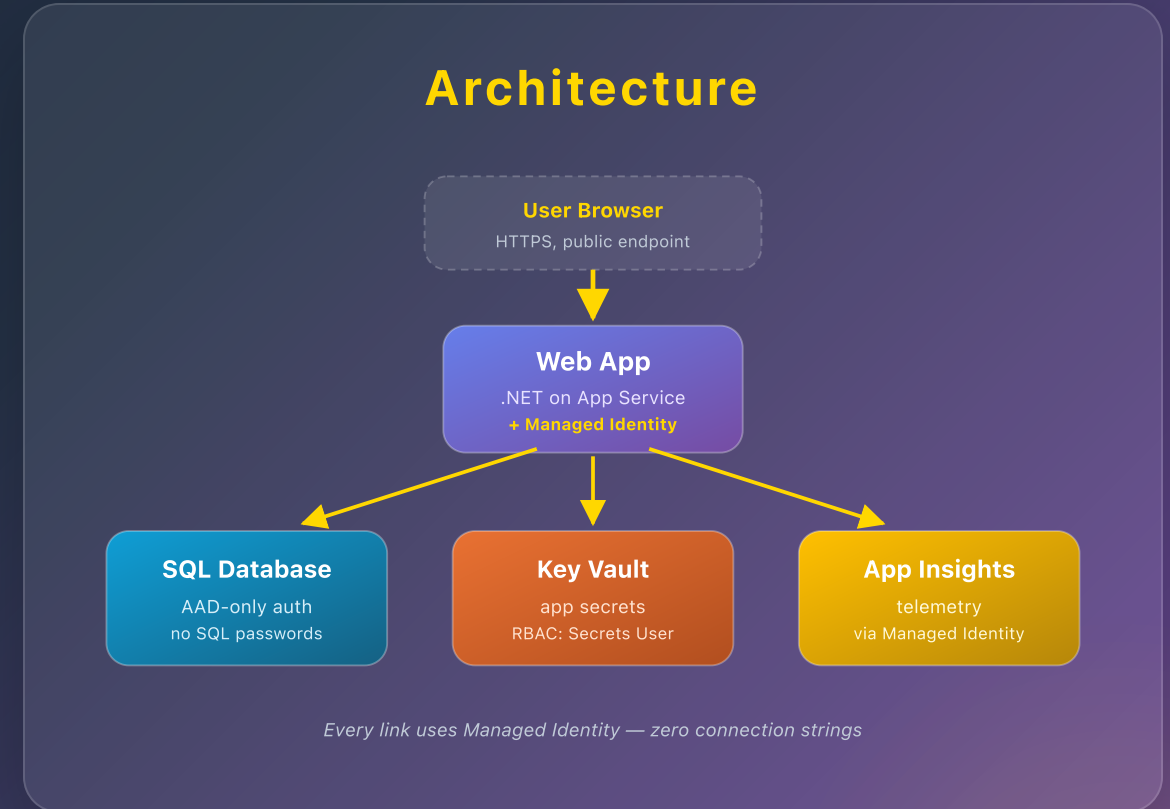
Numbers come straight from the Azure Retail Prices API - same numbers you'll see on the invoice.

What You Build Today

Web App + SQL Database + Key Vault

Resources

- **App Service Plan + Web App** with managed identity
- **SQL Server + Database** with AAD-only auth
- **Key Vault** for app secrets, accessed by RBAC
- **Application Insights** for monitoring and traces
- All connected via the **managed-identity chain**



Zero connection strings

Zero SQL passwords

Lab Roadmap

Five labs across 60 minutes

Lab	Time	What you do
1. Onboarding	10 min	OIDC, RBAC, and GitHub environments
2. Web App + SQL	15 min	Multi-resource deployment
3. Security Deep Dive	10 min	Break security, gate blocks, fix
4. Cost & Architecture	10 min	Cost estimate plus WAF review
5. Drift Detection	5 min	Manual drift, detect, reconcile

Labs 1 and 2 are foundational. Labs 3-5 each stand alone, so abbreviate the last one if time slips.

The Aha Moment

What happens when you break security?

Walk-through (Lab 3)

1. You enable **shared key access** on a Storage Account (anti-pattern)
2. Git-Ape Security Analyzer catches it: **SECURITY GATE: BLOCKED**
3. **Deployment is prevented** - no resource is created
4. You disable shared keys, re-run, gate flips to **SECURITY GATE: PASSED**

Why this matters

Security is enforced, not advisory

The gate is the difference between "we have policies" and "policies actually run."

Same rules in dev and prod

You catch the violation on your laptop, not in a production audit.

Override requires intent

"I accept the security risk" is logged with justification. No silent bypass.

Let's Go!

Open your environment and start Lab 1

1. Open env

Codespaces
Dev Containers
or VS Code

Tools pre-installed in
containers

2. az login

Standard browser
or device code
in Codespaces

Use your sandbox subscription

3. Lab 1

Onboarding
OIDC + RBAC
10 minutes

Foundation for Labs 2-5

If you completed Track 1, you already have the environment ready - jump straight to **az login** and then **Lab 1**.